

Tasks: Tokyo Mate 東京通 | Nearby Map & Navigation 探索附近地圖與導航

Input: Design documents from </specs/003-nearby-map-navigation/>

Prerequisites: [plan.md](#) (required) 、 [spec.md](#) (required for user stories) 、 [research.md](#) 、 [data-model.md](#) 、 [contracts/nearby-map-navigation-contracts.md](#) 、 [quickstart.md](#)

Tests: 本 feature 之 [plan.md](#) 「Testing Plan」 章節已明確要求對應 14 項情境的單元測試，故本 [tasks.md](#) 包含測試任務（非樣板預設，而是依 [plan.md](#) 明確要求納入）。

Organization: 任務依 [spec.md](#) 之 User Story (P1/P1/P2/P3) 分組，確保各 story 可獨立實作與驗證。

⚠ Implementation Authorization: 本 [tasks.md](#) 產生後，仍須待使用者於 [/speckit.analyze](#) 與 Implementation Readiness Gate 完成後，明確批准進入 [/speckit.implement](#)，方可修改 application code (依 [copilot-instructions.md](#) Authorization Boundary) 。

Format: [\[ID\]](#) [\[P?\]](#) [\[Story\]](#) [Description](#)

- **[P]:** 可平行執行（不同檔案、無相依）
- **[Story]:** 對應 [spec.md](#) 的 US1 / US2 / US3 / US4
- 每項任務均含明確檔案路徑

Path Conventions

沿用既有 single-project 結構：[src/](#) (frontend) 、 [api/](#) (Vercel serverless functions，本 feature 不新增 [api/*](#) 路由) 。無 [backend/](#) / [frontend/](#) 分離目錄。

Phase 1: Setup (Shared Infrastructure)

Purpose: 安裝本 feature 唯一新增之 runtime 依賴，並準備新增的 client-side 環境變數位置

- ☒ T001 於 repository root 執行 `npm install leaflet` (並視 TypeScript 型別整合需要以 `npm install -D @types/leaflet`) 安裝本 feature 唯一新增依賴，更新 [package.json](#)
- ☒ T002 [P] 於 [.env.example](#) 新增 `VITE_GEOAPIFY_API_KEY=` 這一行（不含實際值），沿用既有 `GOOGLE_PLACES_API_KEY=` 格式慣例

Checkpoint: 依賴與環境變數位置就緒，可進入 Foundational 階段

Phase 2: Foundational (Blocking Prerequisites)

Purpose: US1 (地圖) 與 US2 (前往此地) 共用的座標可靠性判定 · MUST 先完成

⚠ **CRITICAL:** 本階段完成前 · 任何 User Story 階段不得開始

- ☒ T003 建立 `src/features/nearby/mapEligibility.ts` : 實作 `hasReliableCoordinates()` (內部) 、
`isMapEligible(place)` 、 `isNavigationEligible(place)` 純函式 (座標存在、有限數值、且非 `(0, 0)` 時為 `true`) · 依 `data-model.md` #2 與 `contracts/nearby-map-navigation-contracts.md` #1
- ☒ T004 [P] 建立 `src/features/nearby/mapEligibility.test.ts` : 涵蓋可靠座標、座標缺失、
`(0, 0)` fallback、非有限數值 (`NaN/Infinity`) 四類情境 · 驗證 `isMapEligible` 與 `isNavigationEligible` 回傳一致

Checkpoint: Foundation ready — 可開始 User Story 實作

Phase 3: User Story 1 - 用地圖理解目前位置搜尋出的附近結果 (Priority: P1) 🌀 MVP

Goal: 使用者以目前位置搜尋附近結果後 · 可透過地圖理解這些結果的位置與空間關係 · 且地圖與既有 result cards 呈現同一批結果。

Independent Test: 以目前位置搜尋出具備可靠位置資料的結果 · 可在不涉及選取或導航的情況下 · 單純透過地圖確認結果分佈與既有 result cards 一致。

Tests for User Story 1 ⚠

NOTE: 先撰寫測試並確認失敗 · 再進行實作

- ☒ T005 [P] [US1] 建立 `src/features/nearby/NearbyMap.test.tsx` : 以 `vi.mock('leaflet')` 驗證 `L.map` / `L.tileLayer` (含 Geoapify URL 與 attribution) / `L.marker` 呼叫行為、`places` 變化時 `marker` 集合更新、初次取得非空 `places` 時呼叫 `fitBounds`、`marker` 點擊觸發 `onSelectPlace(id)` 、
(a) 地圖初次建立 / 載入失敗時呼叫一次 `onMapUnavailable` 且不拋出例外； (b) 地圖初次建立成功
後 · 模擬 `tile` 載入或既有地圖執行期間發生無法復原錯誤 (例如 Leaflet 內部錯誤事件) 時 · 同樣呼叫
一次 `onMapUnavailable` 且不拋出例外——此案例與 (a) 分開驗證 · 確保涵蓋「地圖已建立後的使用期
間」可觀察 runtime failure · 而非僅涵蓋初次建立 / 載入失敗；並依 FR-003 / SC-003 以 mock / spy 驗證：
(d) 模擬 Leaflet map 的 `pan` / `moveend` 事件後 · 未呼叫 `requestPlaces` 或任何 `/api/*` fetch mock；
(e) 模擬 Leaflet map 的 `zoom` / `zoomend` 事件後 · 同樣未呼叫 `requestPlaces` 或任何 `/api/*`

fetch

mock，且 marker 集合與既有 `places` 維持不變

- ☒ T006 [P] [US1] 擴充 `src/screens/NearbyScreen.test.tsx`：新增案例涵蓋 (a) 成功搜尋且結果具備可靠座標時渲染地圖、(b) 缺乏可靠座標的有效 result 不出現在地圖但仍是有效卡片、(c) 成功搜尋但回傳 0 筆結果時地圖 0 個 marker、Selected Place 維持未選定、且與搜尋失敗 / 地圖不可用狀態可明確區分；並依 FR-003 / SC-003 新增 integration-level 案例 (d)：於畫面已呈現一批 Nearby results 後，觸發地圖 pan / zoom (透過 `NearbyMap` mock 觸發 `onSelectPlace` 以外的 pan/zoom 回呼或直接呼叫底層 mock 事件)，驗證 `requestPlaces` / 對應 fetch mock 的呼叫次數與操作前相同 (不增加)，且畫面仍呈現同一批既有 Nearby results，不產生第二次搜尋或 marker 集合變化

Implementation for User Story 1

- ☒ T007 [US1] 建立 `src/features/nearby/NearbyMap.tsx`：以 `useRef` + `useEffect` 管理 Leaflet 地圖生命週期 (掛載 / 解構)，以 `L.tileLayer` 掛載 Geoapify raster XYZ tiles (URL 內帶入 `import.meta.env.VITE_GEOAPIFY_API_KEY`) 並附上必要 attribution；依 `places` prop 建立/更新 marker、依 `selectedPlaceId` 呈現非純色彩的選定樣式；marker 點擊呼叫 `onSelectPlace`；地圖初次建立 / 載入，或地圖已建立後使用期間 (例如 tile 載入或既有地圖執行) 發生無法復原錯誤時，均須 catch 並呼叫一次 `onMapUnavailable`，不得使父層 crash；不得因 pan / zoom 呼叫任何 `/api/*` 請求 (依 [contracts/nearby-map-navigation-contracts.md](#) #3)
- ☒ T008 [US1] 擴充 `src/screens/NearbyScreen.tsx`：新增 `selectedPlaceId` (`string | undefined`) 與 `mapUnavailable` (`boolean`) local state；以 `isMapEligible` 過濾 `places` 後傳入 `NearbyMap`；串接 `onSelectPlace` (暫時僅更新 `selectedPlaceId`，完整雙向串接於 Phase 4 完成) 與 `onMapUnavailable` (設定 `mapUnavailable` = `true`，不得設定既有 `error`)
- ☒ T009 [US1] 於 `src/screens/NearbyScreen.tsx` 呈現層新增：當 `mapUnavailable` 為 `true` 時顯示簡短「地圖暫時無法使用」文案 (獨立於既有 `error` / `notice` 呈現)，且不影響 `NearbyResults` 的可讀可操作

Checkpoint: User Story 1 應可獨立完成實作與驗證 (地圖呈現、pan/zoom 不觸發搜尋、地圖不可用時卡片仍可用、空結果與失敗狀態可區分)

Phase 4: User Story 2 - 選定地點後前往此地 (Priority: P1)

Goal: 使用者從搜尋結果中選定一個具備足夠且可靠目的地資訊的地點後，可啟動「前往此地」，交由 Google Maps HTTPS handoff 處理。

Independent Test: 選定一個具備可靠目的地資訊的 `result`，啟動「前往此地」並確認嘗試交由外部地圖 / 導航服務處理，不需要先完成地圖選取流程。

Tests for User Story 2 ⚠

- ☒ T010 [P] [US2] 建立 `src/features/nearby/navigation.test.ts`：驗證 `buildGoogleMapsHandoffUrl`
對合格 `result` 回傳正確 Google Maps HTTPS URL、對不合格 `result` 回傳 `undefined`；驗證 `openNavigationHandoff` 在合格時呼叫 `window.open(url, '_blank', 'noopener,noreferrer')` 並回傳 `{ success: true }`，在不合格、`window.open` 回傳 `null` 或拋出例外時回傳 `{ success: false }` 且
不拋出未捕捉例外
- ☒ T011 [P] [US2] 擴充 `src/features/nearby/NearbyResults.test.tsx`：驗證每張卡片可獨立呼叫
`onSelectPlace`、選定卡片具備非純色彩可辨識標示（例如文字標籤或 `aria-pressed`）、「前往此地」按鈕
僅在 `isNavigationEligible(place)` 為 `true` 時可點擊觸發 `onNavigate`，不合格時不可觸發
- ☒ T012 [P] [US2] 擴充 `src/screens/NearbyScreen.test.tsx`：驗證 `handoff` 成功 / 失敗均保留 `selectedPlaceId` 與既有 `places`；`handoff` 失敗時設定的 `navigationNotice` 訊息與既有 `error`（搜尋失敗）、`notice`（定位失敗）三者訊息可明確區分，不互相覆蓋

Implementation for User Story 2

- ☒ T013 [US2] 建立 `src/features/nearby/navigation.ts`：實作 `buildGoogleMapsHandoffUrl(place)`（僅在 `isNavigationEligible(place)` 為 `true` 時回傳以合格座標與可讀名稱組成的 Google Maps HTTPS URL，否則回傳 `undefined`）與 `openNavigationHandoff(place)`（呼叫 `buildGoogleMapsHandoffUrl`，不合格回傳 `{ success: false }` 且不呼叫 `window.open`；合格時呼叫 `window.open(...)`，依結果/例外回傳 `{ success: true | false }`，不拋出未捕捉例外），依 [contracts/nearby-map-navigation-contracts.md #2](#)
- ☒ T014 [US2] 擴充 `src/features/nearby/NearbyResults.tsx`：新增 `selectedPlaceId`、`onSelectPlace`、`onNavigate` props；每張卡片依 `onSelectPlace(place.id)` 完成選取（不依賴地圖是否已渲染）；選定卡片呈現非純色彩標示；「前往此地」按鈕僅在 `isNavigationEligible(place)` 為 `true` 時顯示為可點擊，
點擊時呼叫 `onNavigate(place)`
- ☒ T015 [US2] 擴充 `src/screens/NearbyScreen.tsx`：新增 `navigationNotice (string | undefined)`

state ; 新增共用 `handleSelectPlace(id)` 同時供 `NearbyMap.onSelectPlace` 與 `NearbyResults.onSelectPlace` 呼叫，確保 `selectedPlaceId` 為 map / card 唯一共用狀態；新增 `handleNavigate(place)` 呼叫 `openNavigationHandoff`，依 `success` 設定或清除 `navigationNotice`，失敗時不得清除 `selectedPlaceId` 或 `places`；呈現層新增 `navigationNotice` 專屬文案區塊，與既有 `error / notice` 明確區分

Checkpoint: User Story 1 與 2 應皆可獨立運作，並共用同一個 `selectedPlaceId`

Phase 5: User Story 3 - 手動輸入地區也能使用地圖與前往此地 (Priority: P2)

Goal: 即使沒有目前位置，使用者手動輸入地區完成搜尋後，仍可使用地圖理解結果位置，並對合格 result 啟動「前往此地」。

Independent Test: 在未授予定位權限的情況下，直接以手動輸入地區完成搜尋，確認可查看地圖並對合格 result 啟動「前往此地」。

Tests for User Story 3 ⚠️

- ☒ T016 [P] [US3] 擴充 `src/screens/NearbyScreen.test.tsx`：模擬未曾使用目前位置、直接以 `handleManualAreaSearch` 完成搜尋，驗證地圖呈現具備可靠座標的結果、且對合格 result 啟動「前往此地」全程不要求或依賴目前位置

Implementation for User Story 3

- ☒ T017 [US3] 檢查並視需要調整 `src/screens/NearbyScreen.tsx` 之 `handleManualAreaSearch`：確保與 `handleUseMyLocation` 共用相同的 `selectedPlaceId` / `navigationNotice` 重置邏輯與相同的 `NearbyMap` / `NearbyResults` 串接方式，不因搜尋來源（手動地區 vs 目前位置）而產生不同行為

Checkpoint: User Story 3 應可獨立驗證（手動地區搜尋不依賴目前位置即可使用地圖與前往此地）

Phase 6: User Story 4 - 定位失敗、地圖不可用或選取來源不同時仍可完成任務 (Priority: P3)

Goal: 定位失敗可恢復手動地區模式；地圖不可用時卡片仍可完成選取與前往此地；map / card 選取一致對應；新搜尋開始時舊內容不被誤認為新結果。

Independent Test: 分別模擬定位被拒絕 / 取消 / 無法取得、地圖載入失敗、從地圖開始選取、以及重新開始新搜尋等情境，確認既有結果與選取狀態均未被破壞。

Tests for User Story 4 ⚠️

- ☒ T018 [P] [US4] 擴充 `src/screens/NearbyScreen.test.tsx` : 模擬定位權限被拒絕 / 使用者取消 / 定位逾時
三種情境，驗證既有搜尋結果不被破壞，且可切換手動地區模式完成搜尋 (既有 `notice` 狀態與新增 `navigationNotice` 不互相干擾)
- ☒ T019 [P] [US4] 擴充 `src/screens/NearbyScreen.test.tsx` (或 `NearbyMap.test.tsx`) : 驗證
`onMapUnavailable` 觸發時，`NearbyResults` 仍可讀、可操作，可直接由卡片完成選取與 (合格時) 啟動
「前往此地」，且不設定既有 `error`
- ☒ T020 [P] [US4] 擴充 `src/screens/NearbyScreen.test.tsx` : 驗證先由 `NearbyMap` marker 點擊選取的地點，會使 `NearbyResults` 對應卡片顯示為目前選定；反向 (先點卡片) 驗證地圖 marker 同步反映同一個
`selectedPlaceId`
- ☒ T021 [P] [US4] 擴充 `src/screens/NearbyScreen.test.tsx` : 驗證開始新一次搜尋
(`handleUseMyLocation`
或 `handleManualAreaSearch`) 時，`selectedPlaceId` 與 `navigationNotice` 立即重置，且
`loading`
為 `true` 期間不呈現上一次搜尋的地圖或結果內容

Implementation for User Story 4

- ☒ T022 [US4] 於 `src/screens/NearbyScreen.tsx` 之 `handleUseMyLocation` 與
`handleManualAreaSearch` 起始處 (送出請求前) 同步呼叫 `setSelectedPlaceId(undefined)` 與
`setNavigationNotice(undefined)`，確保新搜尋轉換行為一致 (依
[data-model.md #3 Search Transition](#))
- ☒ T023 [US4] 確認 `src/features/nearby/NearbyResults.tsx` 既有 `if (loading) return`
<p>正在搜尋附近 地點...</p> 提前 `return` 分支，於 `loading` 為 `true` 期間同時涵蓋地圖區域
(`NearbyMap` 不接收舊
`places` / 不渲染)，避免新舊資料混合呈現

Checkpoint: 四個 User Story 應皆可獨立完成實作與驗證

Phase 7: Polish & Cross-Cutting Concerns

Purpose: 影響多個 User Story 的收尾項目

- ☒ T024 [P] 於 `src/styles/components.css` 新增 `NearbyMap` 容器 responsive 樣式與選定 marker / 卡片的非純色彩視覺標示樣式 (例如額外圖示或框線，而非僅改變顏色)
- ☒ T025 執行既有回歸驗證：於 repository root 執行 `npm run build`、`npx vitest run --pool=threads`，
並依既有 Playwright regression 設定確認 Feature 001 MVP 與 Feature 002 拍照翻譯既有可觀察行為不變

- ☐ T026 依 [quickstart.md](#) 逐一手動驗證 4 個 User Story 之驗證場景，並確認未修改 [specs/001-tokyo-travel-assistant/](#) 或 [specs/002-photo-translate/](#) 內容
- ☐ T027 **[Production Release 前必要安全任務]** 正式上線前，於 Geoapify 帳戶將 Feature 003 使用的 browser-visible [VITE_GEOAPIFY_API_KEY](#) 設定 Origin/HTTP Referrer restriction，只允許已批准的 Tokyo Mate production origin，並記錄可供 Verify 階段確認的完成證據（例如設定畫面截圖或設定摘要，存放於 repository 外部或 [docs/verification/](#) 之非機密紀錄中）；不得將實際 API key、token 或其他 secret credential 寫入 tasks.md 或 repository 任何檔案。本任務不新增 backend proxy、不改變既有 browser-visible Geoapify key 架構、不新增第二 map provider

Dependencies & Execution Order

Phase Dependencies

- **Setup (Phase 1)**：無相依，可立即開始
- **Foundational (Phase 2)**：依賴 Setup 完成；BLOCKS 所有 User Story
- **User Story 1 (Phase 3)**：依賴 Foundational 完成；無其他 Story 相依
- **User Story 2 (Phase 4)**：依賴 Foundational 完成；與 US1 共用 [NearbyScreen.tsx](#) 之 [selectedPlaceId](#) 串接（T015 依賴 T008 已建立的 [selectedPlaceId](#) state），但可獨立驗證「前往此地」核心行為
- **User Story 3 (Phase 5)**：依賴 Foundational、US1（T007-T009）與 US2（T013-T015）完成的地圖 / 導航串接方式，用於確認手動地區路徑行為一致
- **User Story 4 (Phase 6)**：依賴 Foundational、US1、US2 完成，強化既有兩個核心能力的韌性
- **Polish (Phase 7)**：依賴所有已完成的 User Story

User Story Dependencies

- **US1 (P1)**：Foundational 完成後即可開始，無其他 Story 相依
- **US2 (P1)**：Foundational 完成後即可開始；與 US1 共用 [NearbyScreen.tsx](#) 之 [selectedPlaceId](#)，故實務上建議接續 US1 之後實作（T008 完成後再進行 T015），但「前往此地」核心邏輯（T013、T014）本身不依賴地圖是否已存在
- **US3 (P2)**：驗證既有 US1 / US2 行為在手動地區路徑下同樣成立，非新增獨立邏輯
- **US4 (P3)**：強化 US1 / US2 之韌性與一致性，非新增獨立價值主張

Within Each User Story

- 先撰寫測試並確認失敗，再進行實作（T005/T006 早於 T007-T009；T010-T012 早於 T013-T015；以此類推）
- [mapEligibility.ts](#) / [navigation.ts](#) 等純函式模組先於串接 [NearbyScreen.tsx](#) 的任務
- Story 完成（含 Checkpoint 驗證）才進入下一優先序 Story

Parallel Opportunities

- Setup 階段 T002 可與 T001 平行
- Foundational 階段 T004 可與 T003 完成後平行撰寫 (測試依賴函式簽章 · 可先撰寫再對照實作)
- 每個 User Story 內標示 [P] 的測試任務可平行撰寫 (不同檔案)
- US3、US4 之驗證測試任務 (T016、T018-T021) 彼此可平行進行

Parallel Example: User Story 1

```
# 平行撰寫 User Story 1 的測試任務 :  
Task: "建立 src/features/nearby/NearbyMap.test.tsx ( vi.mock('leaflet') ) "  
Task: "擴充 src/screens/NearbyScreen.test.tsx 新增地圖相關案例"
```

Parallel Example: User Story 2

```
# 平行撰寫 User Story 2 的測試任務 :  
Task: "建立 src/features/nearby/navigation.test.ts"  
Task: "擴充 src/features/nearby/NearbyResults.test.tsx"  
Task: "擴充 src/screens/NearbyScreen.test.tsx 新增 handoff 相關案例"
```

Implementation Strategy

MVP First (User Story 1 + User Story 2 · 皆為 P1)

1. 完成 Phase 1 : Setup
2. 完成 Phase 2 : Foundational (CRITICAL · 封鎖所有 User Story)
3. 完成 Phase 3 : User Story 1 (地圖理解)
4. **STOP 並驗證** : 獨立測試 User Story 1
5. 完成 Phase 4 : User Story 2 (前往此地)
6. **STOP 並驗證** : 獨立測試 User Story 2 ; 此時 MVP (地圖理解 + 前往此地) 已可展示

Incremental Delivery

1. 完成 Setup + Foundational → 基礎就緒
2. 加入 User Story 1 → 獨立測試 → 展示 (地圖理解)
3. 加入 User Story 2 → 獨立測試 → 展示 (前往此地 · MVP 完整)
4. 加入 User Story 3 → 獨立測試 → 展示 (手動地區同等能力)
5. 加入 User Story 4 → 獨立測試 → 展示 (韌性與一致性強化)
6. 每個 Story 皆在不破壞先前 Story 的前提下增加價值

Notes

- [P] 任務 = 不同檔案、無相依
- [Story] 標籤將任務對應至 spec.md 之特定 User Story · 利於追蹤

- 每個 User Story 應可獨立完成與測試
- 實作前先確認測試失敗 (TDD 慣例 , 依 plan.md Testing Plan 要求)
- 每個 Checkpoint 停下驗證該 Story 獨立性後 , 才進入下一個 Story
- 避免 : 模糊任務、同檔案衝突、破壞 Story 獨立性的跨 Story 相依
- 本 tasks.md 不構成 implementation 授權 ; 須待 `/speckit.analyze` 與使用者明確批准後方可執行 `/speckit.implement`